

Actividad 3. Diseno, desarrollo e implementacion del visor web

Esta actividad documenta el diseno, desarrollo e implementacion del visor web TerraSight para el convenio CAR Cundinamarca - WWF - Fundacion Natura. Se incluyen los requerimientos funcionales y no funcionales del sistema, los casos de uso por rol de usuario, el diseno visual de la interfaz, la arquitectura tecnica del visor y la implementacion final con sus modulos.

3.1. Diseno del sistema e interfaz de usuario

El diseno del sistema sigue el principio de separacion entre capa de datos (PostgreSQL/PostGIS), capa de aplicacion (Next.js API routes) y capa de presentacion (React con Tailwind CSS). La interfaz se organiza en modulos funcionales independientes que comparten un sistema de diseno comun (tokens, primitives, layout) y un sidebar persistente que refleja la matriz de acceso por rol.

3.1.1. Requerimientos funcionales y no funcionales del diseno del sistema

Requerimientos funcionales

Los requerimientos funcionales se derivan directamente de las historias de usuario implementadas en el repositorio (HU-AD, HU-AA, HU-CO, HU-TC, HU-IC, HU-MO, HU-CA).

Codigo	Requerimiento	Modulo	Rol minimo
RF-01	Iniciar sesion con email y contrasena (NextAuth Credentials, JWT)	/login	Todos
RF-02	Recuperar contrasena por email (pendiente, TODO login-form.tsx:135)	/login	Todos
RF-03	Listar y buscar predios por nombre, codigo o componente	/predios	Todos
RF-04	Crear, editar y eliminar predios (alta, baja, modificacion)	/predios + /[id] + /nuevo	ADMIN, GESTOR
RF-05	Listar y buscar quebradas con filtro por municipio	/quebradas	Todos
RF-06	Crear, editar y eliminar quebradas con estado editable	/quebradas + /nuevo	ADMIN, GESTOR
RF-07	Listar intervenciones con filtro por componente y municipio	/intervenciones	Todos
RF-08	Ver ficha detallada de intervencion con geometria en mapa, timeline de avances y edicion de % de avance	/intervenciones/[id]	Todos (lectura) / ADMIN, GESTOR (escritura)
RF-09	Cambiar estado de intervencion (Pendiente / En ejecucion / Finalizada)	/intervenciones	ADMIN, GESTOR

Codigo	Requerimiento	Modulo	Rol minimo
RF-10	Visualizar mapa interactivo con capas (predios, quebradas, municipios, veredas, parques, reservas)	/mapa	Todos
RF-11	Realizar busqueda de predios por nombre/codigo en el visor	/mapa	Todos
RF-12	Ejecutar analisis espacial buffer sobre quebrada o propuesta	/analisis	ADMIN, ANALISTA
RF-13	Consultar matriz componente x municipio con hectareas y # propuestas	/analisis	ADMIN, ANALISTA
RF-14	Calcular interseccion por bounding box (bbox)	/analisis	ADMIN, ANALISTA
RF-15	Generar 10 reportes operativos (R1 a R10) con export CSV en formato es-CO	/reportes + /api/reportes	ADMIN, ANALISTA
RF-16	Ver alertas clasificadas (critica, advertencia, informativa)	/alertas	ADMIN, ANALISTA
RF-17	Gestionar usuarios (alta, baja, cambio de rol)	/admin/usuarios	ADMIN
RF-18	Ver auditoria de accesos (LOGIN_OK, LOGIN_FAIL, LOGOUT, ACCESS_DENY)	/admin/auditoria	ADMIN
RF-19	Gestionar catalogos: componentes, acciones, municipios, veredas, propietarios, microcuencas, beneficiarios	/catalogos + 5 sub-paginas	ADMIN
RF-20	Ver modulo de monitoreo: obras de captacion, estaciones limnimetricas, bebederos, tanques, paneles solares con gestion de beneficiarios	/monitoreo + /[id] (punto)	Todos (lectura) / ADMIN, GESTOR (escritura)

Requerimientos no funcionales

Codigo	Requerimiento	Categoria	Cumplimiento
RNF-01	Tiempo de respuesta API < 500 ms para listados (con withFallback a demo data si BD cae)	Performance	repository.ts: withFallback por query
RNF-02	SRID mixto (4326 propuestas + 4686 predios) con conversion explicita geography para distancias en metros	Geoespacial	ST_Distance(a::geography, b::geography)
RNF-03	Indices GiST en todas las columnas geometricas (predios, propuestas, geom)	Geoespacial	CREATE INDEX ... USING GIST
RNF-04	Autenticacion JWT con sesion httpOnly, estrategia Credentials provider (email + password)	Seguridad	NextAuth 5.0.0-beta.25
RNF-05	Hash de contraseñas con bcryptjs (puro JS, sin compilacion nativa)	Seguridad	bcryptjs 2.4.3
RNF-06	Middleware de proteccion de rutas (excepto /login, /api/auth, assets)	Seguridad	src/middleware.ts (Edge runtime)

Codigo	Requerimiento	Categoria	Cumplimiento
RNF-07	Validacion declarativa de formularios con safeParseForm (sin Zod ni Yup)	Validacion	lib/validation.ts
RNF-08	Roles ADMIN / ANALISTA / GESTOR con matriz de acceso server-side (requireRole) y filtro client-side (sidebar)	Autorizacion	lib/auth-guard.ts
RNF-09	Auditoria safe-fail: si falla la escritura, no rompe la operacion del usuario	Auditoria	lib/audit.ts: safe-fail writes
RNF-10	Compatibilidad con Chrome, Firefox, Edge (modernos) en escritorio; tablet opcional	Compatibilidad	sin hacks de IE11; Tailwind 4
RNF-11	Codificacion CSV es-CO con BOM y separador ; (Excel-en-espanol)	i18n	lib/csv.ts (50 lineas, sin deps)
RNF-12	Tipografia Hanken Grotesk y paleta institucional (emerald #006d37, ocean #2f6388, ochre #944a00)	UX/UI	globals.css @theme
RNF-13	Cobertura de tests: 156 unit (Vitest) + 10 e2e (Playwright); objetivo > 80% en pure functions	Calidad	tests/unit + tests/components + tests/e2e
RNF-14	CI automatizado con typecheck + lint + test en cada push a main	DevOps	.github/workflows/ci.yml
RNF-15	Documentacion inline con bloques ASCII y JSDoc; commits con scope semantico (feat/fix/chore)	Mantenibilidad	Conventional commits + READMEs

3.1.2. Desarrollo de casos de uso

El sistema identifica tres roles principales (Administrador, Analista, Gestor) con permisos diferenciados segun la matriz de acceso final consolidada. A continuacion se documentan los casos de uso principales, sus actores y disparadores.

Actores del sistema

- **ADMIN** (Ana Maria, perfil administrador del convenio): acceso total al sistema, gestion de usuarios, catalogos, configuracion y auditoria.
- **ANALISTA** (perfil tecnico): consulta de datos, generacion de reportes, analisis espacial y alertas. Sin acceso a Monitoreo, Configuracion, Auditoria.
- **GESTOR** (perfil de campo): actualizacion de estados de intervencion y carga de datos de monitoreo. Sin acceso a Analisis, Reportes, Alertas, Auditoria, Catalogos, Configuracion.

Casos de uso principales

ID	Nombre	Actor	Precondicion	Flujo principal	Postcondicion
CU-01	Iniciar sesion	Todos	Usuario registrado en sgs_adm_usuario	1. Ingresa email + password en /login. 2. NextAuth valida con bcryptjs. 3. Si OK, setea cookie	Sesion activa, redirige a destino

ID	Nombre	Actor	Precondicion	Flujo principal	Postcondicion
				httpOnly JWT. 4. Middleware redirige a /dashboard o ruta original.	
CU-02	Registrar nuevo predio	ADMIN, GESTOR	Sesion activa con rol permitido	1. Navega a /predios/nuevo. 2. Completa form (nombre, cedula, nucleo, propietario, vereda, area, lat/lon, observaciones). 3. safeParseForm valida. 4. crearPredioAction ejecuta INSERT. 5. revalidatePath /predios.	Predio creado, redirige a /predios/[id]
CU-03	Visualizar ficha de intervencion	Todos	Sesion activa	1. Desde /intervenciones hace click en fila. 2. /intervenciones/[id] carga IntervencionCompleta via getIntervencionCompleta(id). 3. Renderiza mapa mini (Leaflet con GeoJSON), datos basicos, timeline de avances, form de avance editable si rol lo permite.	Ficha visible con mapa + timeline
CU-04	Registrar avance de intervencion	ADMIN, GESTOR	Sesion activa con rol permitido	1. En /intervenciones/[id] mueve slider 0-100 y escribe nota. 2. actualizarAvanceIntervencionAction ejecuta INSERT en sgs_pro_propuesta_avance. 3. Si 100%, sincroniza estado a Finalizada. 4. router.refresh() repinta timeline.	Nuevo row en timeline, avance actualizado
CU-05	Ejecutar analisis buffer	ADMIN, ANALISTA	Sesion activa con rol permitido	1. En /analisis elige objetivo (quebrada o propuesta) + radio en metros. 2. POST /api/analisis/buffer con ST_DWithin. 3. Resultados renderizados como tabla + GeoJSON en mapa.	Listado de features en buffer
CU-06	Generar reporte R7 (infraestructura por municipio)	ADMIN, ANALISTA	Sesion activa con rol permitido	1. En /reportes selecciona R7 del selector. 2. ejecutarReporteR7() con subqueries pre-agregadas (evita CROSS JOIN). 3. Tabla renderizada. 4. Boton Export CSV: GET /api/reportes?tipo=R7 con header text/csv;charset=utf-8.	Reporte visible, CSV descargado
CU-07	Editar componente del modelo BDG	ADMIN	Sesion activa ADMIN	1. En /catalogos edita nombre del componente. 2. actualizarComponenteAction valida con COMPONENTES_VALIDOS whitelist. 3. UPDATE con ON CONFLICT (UNIQUE) manejo. 4. revalidatePath /catalogos.	Componente actualizado, UI refresca
CU-08	Asociar beneficiario a	ADMIN, GESTOR	Sesion activa con rol permitido	1. En /monitoreo expande fila del punto. 2. Selecciona beneficiario de	Beneficiario linkeado al

ID	Nombre	Actor	Precondicion	Flujo principal	Postcondicion
	punto de monitoreo			listBeneficiariosDisponiblesByPunto. 3. asociarBeneficiarioAction ejecuta INSERT en sgs_rel_propuesta_punto_usuario con ON CONFLICT DO NOTHING.	punto
CU-09	Verificar intentos de acceso denegado	ADMIN	Sesion activa ADMIN	1. Usuario con rol ANALISTA intenta GET /admin/usuarios. 2. requireRole(["ADMIN"]) detecta discrepancia. 3. auditAccessDeny registra en sgs_adm_auditoria_acceso. 4. redirect /dashboard?denegado=1.	Row en auditoria, redirect a dashboard
CU-10	Crear cuenta de usuario	ADMIN	Sesion activa ADMIN	1. En /admin/usuarios hace click en Nuevo. 2. Completa email + nombre + password + rol. 3. crearUsuarioAction valida + hashea password con bcryptjs. 4. INSERT en sgs_adm_usuario.	Usuario creado, aparece en tabla

3.1.3. Diseño visual y estructura de interfaz

La interfaz sigue la identidad "TerraSight Intelligence" con paleta institucional CAR Cundinamarca - WWF - Fundacion Natura, tokens centralizados en `globals.css` y primitives accesibles de Radix UI. El layout se compone de sidebar fijo a la izquierda, topbar superior con buscador y badges de alerta, y area de contenido principal con scroll vertical.

Paleta cromatica

emerald #006d37	ocean #2f6388	ochre #944a00
ink #1a202c	muted #4a5568	surface #f7fafc

Tipografia y primitives

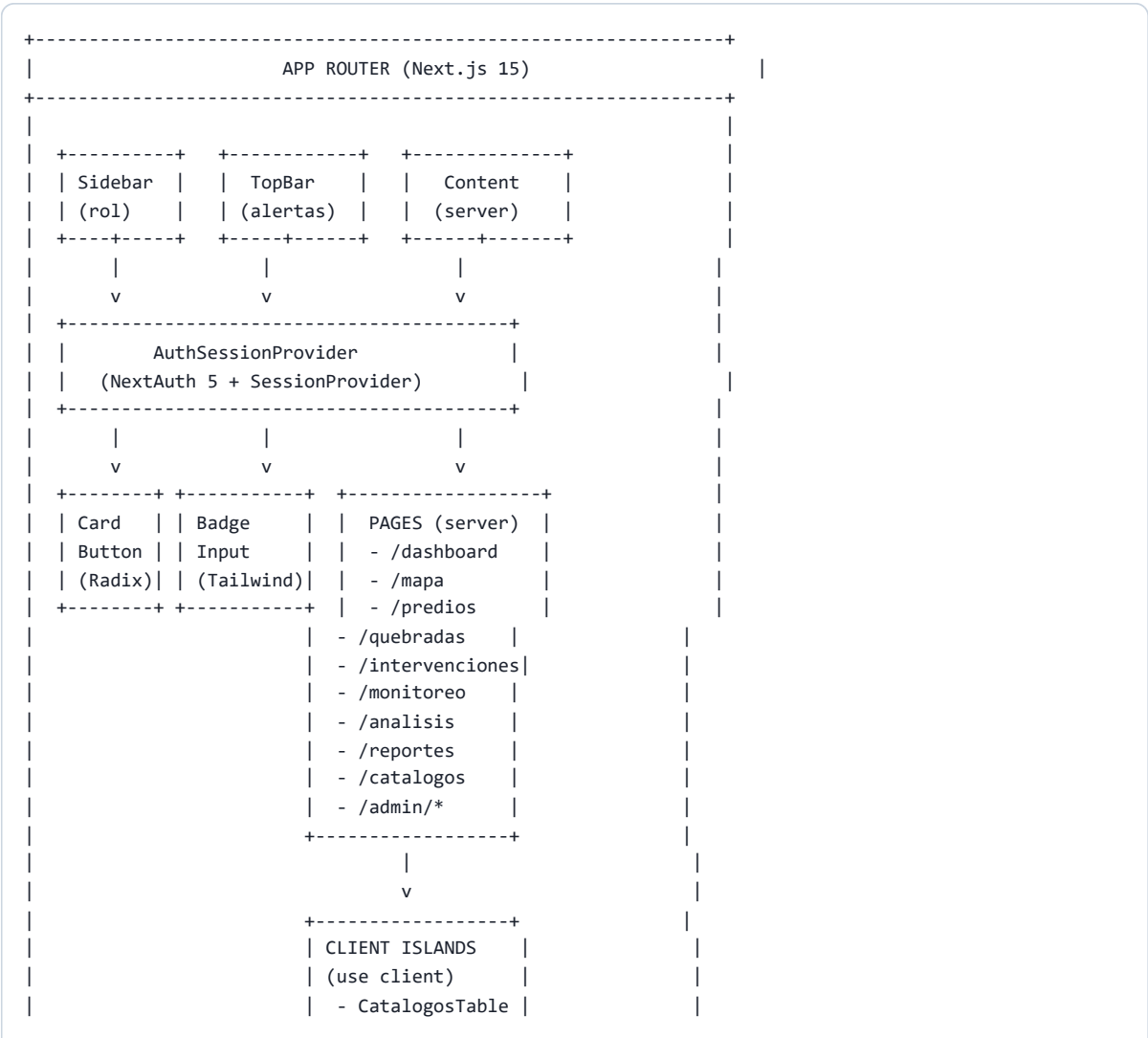
- **Tipografia:** Hanken Grotesk (cargada via @fontsource), con fallback a Georgia para el caso academico PDF.
- **UI primitives:** components/ui/ — Card, Badge, Button, Input basados en Radix UI + class-variance-authority + tailwind-merge.
- **Layout primitives:** components/layout/ — Sidebar (client, filtra por rol), TopBar (search + alertas), ModulePlaceholder (para paginas en construccion).
- **Iconografia:** lucide-react 0.469.0 (iconos open source ISC).

Estructura del sidebar y matriz de visibilidad

El sidebar muestra hasta 11 modulos principales, con visibilidad por rol. Esta es la matriz consolidada tras los sprints 7-12.

Modulo	URL	ADMIN	ANALISTA	GESTOR
Inicio	/	Si	Si	Si
Mapa 2D / 3D	/mapa	Si	Si	Si
Dashboard	/dashboard	Si	Si	Si
Predios	/predios	Si	Si	Si
Quebradas	/quebradas	Si	Si	Si
Intervenciones	/intervenciones	Si	Si	Si
Monitoreo	/monitoreo	Si	NO (server acepta read-only)	Si
Analisis Espacial	/analisis	Si	Si	NO
Reportes	/reportes	Si	Si	NO
Alertas	/alertas	Si	Si	NO
Catalogos	/catalogos	Si	NO	NO
Configuracion	/configuracion	Si	NO	NO

Diagrama de arquitectura de la interfaz



	- MapaMini		
	- EstadoDrop		
	- AvanceForm		
	+-----+		
+-----+			
CAPA DE DATOS (PostgreSQL 16)			
sgs_com_componente	sgs_com_accion	sgs_pro_propuesta	
sgs_pro_predio	sgs_pre_usuario	sgs_adm_usuario	
sgs_adm_auditoria_acceso	sgs_rel_propuesta_punto_usuario		
+-----+			

3.2. Desarrollo del visor web

El visor web es la pieza central de la plataforma TerraSight. Consume datos geoespaciales en formato GeoJSON desde la API de Next.js y los renderiza sobre Leaflet en el navegador. La arquitectura del visor evita servicios intermediarios (WMS/WFS) y mantiene el renderizado 100% client-side, con datos que se sirven como JSON desde la misma aplicacion Next.js.

Stack del visor

- **Next.js 15.1.6** — App Router con Server Components por default y Client Islands ("use client") solo donde se requiere estado o eventos.
- **React 19.0.0** — UI declarativa con concurrent rendering.
- **react-leaflet 5.0.0** — binding declarativo de Leaflet para React, cargado via `dynamic()` con `ssr: false` para evitar el error de Leaflet en SSR.
- **leaflet 1.9.4** — motor de rendering, ~40 KB minificado, BSD-2-Clause.
- **@types/leaflet 1.9.16** — tipos para TypeScript.
- **TypeScript 5.7.3** — strict mode en todo el visor.
- **Tailwind CSS 4.0.0** — utility-first styling para los controles del mapa (no afecta los tiles).

Componentes del visor

Componente	Tipo	Responsabilidad
components/map/leaflet-map.tsx	Server boundary	Wrapper con <code>dynamic()</code> + <code>ssr: false</code> para evitar hydration mismatch
components/map/map-client.tsx	Client (default export)	Orquestador: MapContainer, capas, basemaps (osm/topo/satellite), state, callbacks
components/map/map-layers-panel.tsx	Client	Toggle de 9 capas: predios, quebradas, municipios, veredas, rios, parques, reservas
components/map/map-tools.tsx	Client	Tools: medir, dibujar, zoom in/out, recenter (HU-AA-04 pendiente)
components/map/map-legend.tsx	Client	Leyenda activa segun capas y componente seleccionado
components/map/map-compass.tsx	Client	Rosa de los vientos con orientacion N/S

Componente	Tipo	Responsabilidad
components/map/map-region-labels.tsx	Client	Labels permanentes para los municipios principales del viewport
components/map/map-search-bar.tsx	Client	Buscador de predios con autocompletado

SRID mixto y manejo de geometrias

El visor maneja tres tipos de geometria almacenadas en sub-tablas separadas segun el modelo BDG, todas con SRID 4326 (WGS 84):

- **sgs_pro_propuesta_punto** — geometry(Point, 4326). Render con <Marker> + icono coloreado segun tipo_punto.
- **sgs_pro_propuesta_linea** — geometry(LineString, 4326). Render con <Polyline>.
- **sgs_pro_propuesta_poligono** — geometry(Polygon, 4326). Render con <Polygon> + popup de area en ha.

SRID mixto real: los predios usan SRID 4686 (MAGNA-SIRGAS, datum oficial colombiano) y las propuestas usan SRID 4326 (WGS 84, datum GPS). El visor convierte implicitamente via Leaflet (que trabaja en WGS 84) y el backend usa : :geography para calculos geodesicos correctos en metros.

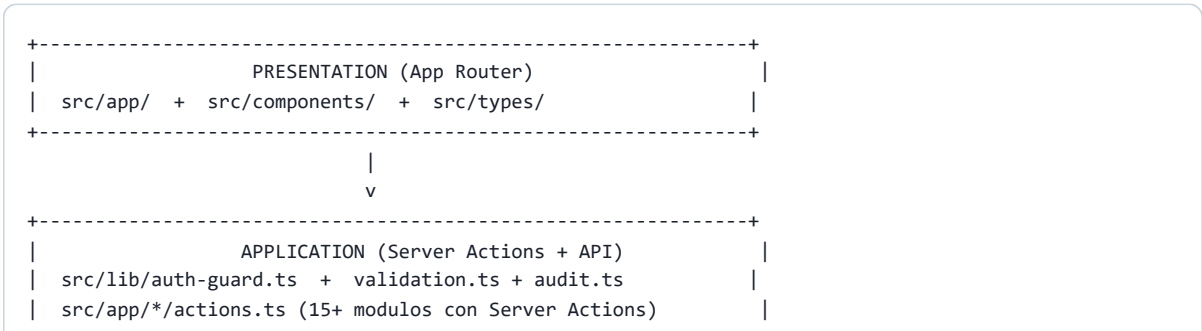
Flujo de datos del visor

1. El usuario navega a /mapa. El server component ejecuta getPrediosGeoJSON() + getQuebradasMini() en paralelo (Promise.all) sobre postgres-js.
2. PostGIS ejecuta ST_AsGeoJSON(geom) y devuelve un FeatureCollection.
3. El cliente renderiza las features con <GeoJSON> o <Marker> segun tipo.
4. El usuario interactua con capas, herramientas o busqueda. Las mutaciones (crear/editar predios) van a /predios/* (Server Actions) o /api/* (route handlers).

3.3. Implementacion y funcionalidad del sistema

La implementacion del sistema sigue una arquitectura limpia en capas con responsabilidades bien delimitadas. El codigo fuente esta organizado en el directorio platform/ del repositorio, con aproximadamente 4.500 lineas de TypeScript y 10 archivos SQL de migracion (3.5k lineas SQL).

Arquitectura de la implementacion



Patron de Server Actions

Todas las mutaciones siguen el mismo patron replicado en 15+ modulos:

1. **Verificacion de rol** con `await requireRole([...])` — red de seguridad server-side que bloquea el 100% de accesos no autorizados.
2. **Validacion declarativa** con `safeParseForm(formData, schema)` — devuelve `{ok, data}` o `{ok: false, message, field}`.
3. **Ejecucion** de la funcion de repository correspondiente, con `try/catch` para capturar errores especificos (23505 UNIQUE, 23503 FK, 23514 CHECK).
4. **Revalidacion** con `revalidatePath("/modulo")` para invalidar cache del App Router.
5. **Mensaje util** devuelto al cliente con explicacion en espanol (nunca el codigo de error crudo de Postgres).

Migraciones SQL

El sistema cuenta con 9 archivos de migracion idempotentes en `platform/scripts/db/init/`:

Archivo	Proposito
00-truncate.sql	Vaciar BD (solo dev)
01-schema.sql	Modelo BDG completo (25+ tablas, geometrias, FK, CHECK, comentarios)
02-datos-ejemplo.sql	Seed demo (CAR Cundinamarca, 11 predios, 151 propuestas, 4 puntos de monitoreo)
03-auth-schema.sql	Tablas <code>sgs_adm_*</code> (usuarios, roles, auditoria de acceso)
04-intervencion-estado.sql	Columna estado en <code>sgs_pro_propuesta</code> + CHECK + backfill + indice
05-catalogos-unique.sql	UNIQUE constraints + trigger <code>sgs_com_set_updated_at()</code> reusable
06-propuesta-avance.sql	Tabla <code>sgs_pro_propuesta_avance</code> con FK CASCADE + CHECK 0-100 + backfill
07-monitoreo-punto.sql	Timestamps en <code>sgs_pro_propuesta_punto</code> + trigger reutilizado
08-cat-secundarios.sql	UNIQUE en 5 catalogos secundarios + timestamps
09-propuesta-avance-es-backfill.sql	Columna <code>es_backfill</code> en tabla de avances (para queries con LATERAL)

Calidad y cobertura de tests

- **156 tests unitarios (Vitest 4.1.10)** cubriendo pure functions: `validation.ts` (35), `csv.ts` (24), `db.ts` (35), `utils.ts` (36), `estado-dropdown.tsx` (10), `login-form.tsx` (16). Cobertura > 95% en los archivos objetivo.
- **10 tests e2e (Playwright 1.61.1)** verificando flujos criticos: redireccion sin auth, render de formularios, deep-link con `callbackUrl`, mensajes de error de credenciales.
- **CI automatizado** (`.github/workflows/ci.yml`) con servicio `postgis/postgis:16-3.4`, migrations automaticas via `scripts/ci-migrate.sh`, typecheck, lint, unit tests, e2e tests.

